

24 march 2024
Tuesday

Blog
(Interview Based class)
Re: watch

Date _____

Page No. _____

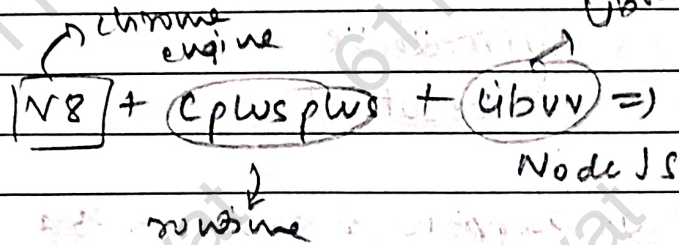


Weekend class (14 march)

Node JS Internal Architecture

⇒ Main Thread

19:07 → class start



libuv

↳ Top level code

↳ Import statements

↳ Event callbacks Register

↳ Event loop

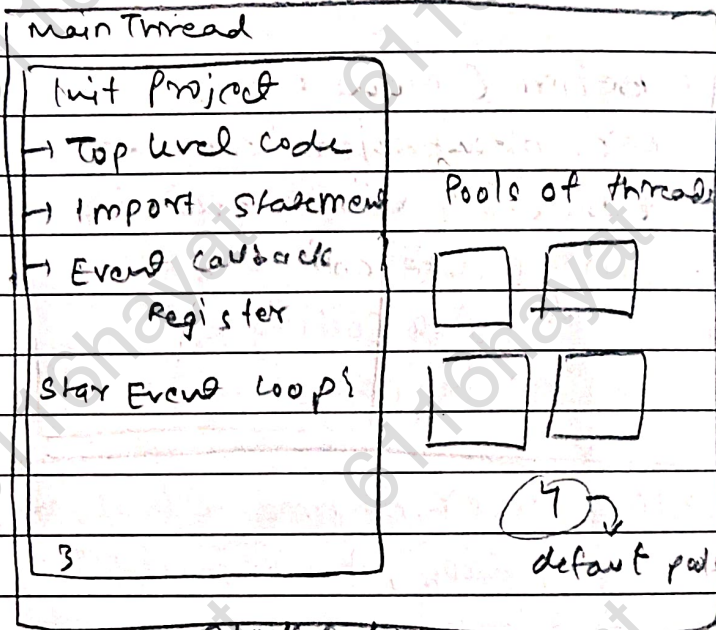
(i) Expired callbacks

(ii) IO Polling

(iii) setImmediate

(iv) close callbacks()

Process



25 march 2024
Wednesday

1:00:00

~~process, env, uv, threadpools~~

process, env, uv.

no. of pools

THREADPOOL-SIZE = 4;

This is how you increase or decrease the size of thread pool workers.

Event loops {

Expired callbacks()

IO Polling()

setImmediate()

close callbacks()

By default, it is 4

Depends on your CPU core

check if anything pending?

No → exit

Yes continue

11:18:00 → Quick

Extra

↳ Post Regularly on Tw

4.

1:35:00 → class ends



Q setTimeout - ②
setImmediate - ①

Q What is work Immediate?

Q What is work of Process.

nextTick() ?
(real time analogy)

code

`setTimeout(() => log("time out"), 0);`

`setImmediate(() => log("immediate"), 0);`

Articles Research

Output →

Immediate
Timeout

⇒ REPL :

Read - Eval - Print - Loop

This happens because of the changes made in Node.js version 20.

• Interactive programming environment

• let's us run JS code line by line directly in terminal

→ Before (Node < 20)

timers phase were running twice per loop iteration

(i) Before poll phase (10 polling)

(ii) After poll phase

Read → takes input

Eval → evaluates

Print → shows the result

Loop → waits for next input

• If a timer became due during poll phase, it would still run immediately in some loop iteration (after poll)

simple

JavaScript Playground
inside terminal

The order was less predictable depend on timing

Q Why setTimeout &

setImmediate are executed differently or

setImmediate is executed first than setTimeout()

output →

timeout

Immediate

OR

Immediate
Timeout



→ Now (Node 20+) with libuv

(i) Timers no longer run before poll

(ii) Timers run only after the poll phase

poll → timers → check (Immediate)

→ Key

- poll finishes
- setImmediate runs (check phase)
- then timers run

So, setImmediate gets a priority edge over times

Output →

Immediate
timeout

Q What is zalgo?

- zalgo is a concept often used in JS discussions to describe control flow caused by mixing synchronous & asynchronous execution in callbacks.

Simple

- A fn is 'zalgo', if it runs synchronously & sometimes asynchronously, making behaviour hard to predict

Analogy

Imagine ordering food

(i) Sometimes, immediately at counter

(ii) have to wait to get delivered

Q What is setImmediate fn?

- A function of Node.js
- schedules a callback to run asynchronously but after the current execution finishes & after I/O events
- Typically before `setTimeout(fn, 0)` in many cases.

Simple Idea:

Run this fn on the next iteration of event loop, right after I/O callbacks.



→ Diff b/w `setImmediate` & `setTimeout`

(i) `setImmediate()`

Runs after the current poll phase finished

- It is tied to check phase of event loop
- It does not care about time
- executed as soon as the current I/O work is done

(ii) `setTimeout()`

Runs after at least ms milliseconds

- It is tied to the timers phase
- It depends on:
 - ↳ minimum delay
 - ↳ & whether the event loop is free to execute

poll (I/O work)



check (`setImmediate`)



timers (`setTimeout`)

Q What is `process.nextTick()`

- It is a function that schedules a callback to run:
- Immediately after the current operation finishes but before the event loop continues.

→ Execution order

Top level code



`process.nextTick()`



event loop phase

(timers / poll / check)

→ Why it exists

1. Run code asap (but async) lets us defer execution without waiting for event loop

2. Fix "Zalgo"

(sync & async consistency)

3. Error handling & cleanup

→ Final key takeaway

`process.nextTick()` is for ultra-high-priority async execution



Use it when:

- you need something to run immediately after current code
- you want consistent async behaviour

Avoid it:

you're looping or scheduling repeatedly (can block everything)